

Research Answers

Name: Manzezulu Mazibuko

Task: Django - Sticky Notes Application, Part 1

1. How HTTP applications preserve state across requests

HTTP is a stateless protocol, each request a browser sends is handled by the server as an isolated event, with no built-in memory of previous requests from the same client. To create an illusion that represents continuous "session" - such as staying logged in while moving between pages - applications rely on a combination of cookies and server-side session storage.

When a user authenticates, the server creates a session record (in Django, stored by default in the database via the `'django.contrib.sessions'` app) and generates a unique session ID. This ID is sent back to the browser as a cookie, which the browser automatically reattaches to every subsequent request to that domain. On each request, the server looks up the session ID in its session store to retrieve data linked to that user, such as their authentication status, permissions, and any temporary data. Because the session ID itself is just a random token, sensitive data is kept server-side rather than in the cookie, reducing the risk of tampering.

Django's authentication system builds on this session framework, once `'django.contrib.auth.login()'` is called, the user's ID is stored in the session, and `'AuthenticationMiddleware'` uses it on every request to attach `'request.user'`. Session cookies can also be configured to expire, be marked `'HttpOnly'` and `'Secure'`, and use `'SameSite'` attributes to guard against cross site attacks, which is an important security consideration when managing authenticated state over an inherently stateless protocol.

2. Performing Django database migrations to a server based relational database (MariaDB)

By default, a new Django project uses SQLite, a lightweight, file-based database well-suited to development. Moving to a server based relational database such as MariaDB for production requires a few configuration changes, but the migration workflow itself (`'makemigrations'` and `'migrate'`) stays the same, since Django's ORM abstracts away the underlying SQL dialect.

First, MariaDB must be installed and a database and user created for the project. Next, a Python database adapter needs to be installed, such as `'mysqlclient'`, so Django can communicate with MariaDB over its wire protocol. In `'settings.py'`, the `'DATABASES'` dictionary is then updated so `'ENGINE'` points to `'django.db.backends.mysql'` (which MariaDB is compatible with), and `'NAME'`, `'USER'`, `'PASSWORD'`, `'HOST'`, and `'PORT'` are set to match the new database's credentials, usually pulled from environment variables rather than hardcoded for security.

Once the connection is configured, running `'python manage.py migrate'` applies all existing migrations, including Django's built-in app migrations and the project's own, against the new MariaDB database, creating the required tables and constraints from scratch. If data already exists in the old SQLite database, it can be exported with `'dump data'` and reloaded into MariaDB with `'load data'` after migrating, since a direct migration only moves the schema, not existing data.